

Website themes

Theme customization

HTML themes are MiniJinja templates. A theme directory contains a required `layout.html` file and optional `partials/`, `styles/`, and `scripts/` directories:

```
themes/my-theme/  
  layout.html  
  partials/  
  styles/main.css  
  scripts/main.js
```

Select a local theme with:

```
html_theme = "themes/my-theme"
```

Use `calepin.toml` for ordinary site branding and navigation changes. Create a local HTML theme when you need to change the HTML shell, top bar, sidebar, table of contents, page navigation, CSS, or JavaScript.

Website builds pass site data into HTML themes. The bundled `calepin-website` theme uses this data for sidebar navigation, previous and next page links, table of contents, metadata, the top-bar brand link, and the GitHub link. Local theme templates can access:

- `site.nav`
- `site.nav_sections`
- `site.navbar_left`
- `site.navbar_center`
- `site.navbar_right`
- `site.languages`
- `site.translations`
- `site.language`
- `site.toc`
- `site.title`
- `site.description`
- `site.base_url`
- `site.logo`
- `site.logo_alt`
- `site.home_url`
- `site.github_url`
- `site.current_url`
- `site.page_title`
- `snippets.css.theme`
- `snippets.css.code`
- `snippets.css.widgets`
- `snippets.js.copy_code`
- `snippets.js.language_picker`
- `snippets.js.theme_toggle`
- `snippets.typst.code_block`

Entries in `site.nav`, `site.nav_sections`, and the navbar regions expose `href`, `label`, `label_html`, `active`, and `widget`. Link entries have `widget = none`. Widget entries preserve the configured

string, so a custom theme can invent names such as `search` or `profile-menu` and render them by checking `item.widget`. The bundled themes understand `widget = "theme"` and `widget = "language"`.

Bundled snippets are available to local HTML themes through the `snippets` object. For example, include reusable base styling, widget styling, code/output styling, and widget behavior with:

```
<style>{{ snippets.css.theme }}</style>
<style>{{ snippets.css.code }}</style>
<style>{{ snippets.css.widgets }}</style>
<script>{{ snippets.js.copy_code }}</script>
<script>{{ snippets.js.language_picker }}</script>
<script>{{ snippets.js.theme_toggle }}</script>
```

`snippets.css.theme` is the shared visual base used by the bundled single-document HTML theme, website theme, and academic theme. It owns common typography, heading scale, Pico primary colors, accent variables, code/output variables, figure defaults, and global document defaults. The bundled themes keep only the layout decisions that are genuinely different, such as sidebars, top bars, table-of-contents placement, profile widgets, and the single-document view switcher.

`snippets.css.widgets` and the matching JavaScript snippets keep interactive controls consistent across themes:

- `snippets.js.theme_toggle` enhances controls marked with `data-calepin-theme-toggle`
- `snippets.js.language_picker` enhances selects marked with `data-calepin-language-picker`

Custom themes should prefer these snippets before copying bundled theme CSS or JavaScript. That keeps dark-mode controls, language pickers, code blocks, and base typography aligned with future Calejpin releases.

PDF themes are Typst files. Their source is inserted after Calejpin's executable-fence rules and before each page source, so they can add show rules for paged output without changing the original `.typ` files. Calejpin also stages reusable Typst snippets under `./calepin/snippets/typst/`; the bundled `calepin-pdf` theme imports `code-block.typ` from there. A minimal theme can replace the default source-block styling:

```
#import "./calepin/snippets/typst/code-block.typ": code-block

#show raw.where(block: true): it => {
  if sys.inputs.at("calepin-target", default: "paged") == "html" {
    it
  } else {
    code-block(it)
  }
}
```

Source and PDF views

The bundled website theme includes a view switcher for rendered HTML, source, and PDF.

The source view is powered by a JSON script embedded in each generated HTML page. The PDF view expects the matching `.pdf` output generated by the default website build.

If you build with `--format html`, the PDF files are not generated. In that mode, the theme can still render HTML and source views, but the PDF view will not have a matching file.